AURA

AUTONOMOUS UNIVERSITY RESPONSE AND ACTION

Project Progress Review

Second presentation study document

Department of Computer Science & Engineering

TKM College of Engineering | Java 17 desktop application

Team

Ringshit

Nirmal

Rehandeep

Adil

Nafee

Progress position: 70% complete | 30% planned work

Prepared from source code, SQL, tests, and engineering documentation

Executive Summary

What AURA solves and where the build stands

AURA is a campus issue and suggestion governance platform. It gives students a trusted way to submit anonymous reports, follow their own tickets, and support important issues through hype votes. Administrators triage the shared queue, move tickets through a five-state lifecycle, and attach resolution notes.

OBJECTIVE

Convert informal campus complaints into traceable, prioritised, and auditable work items while preserving student anonymity.

PROBLEM

Students lack a single channel for issues and suggestions; administrators lack structured prioritisation, status history, and accountability.

CURRENT POSITION

70% complete: core authentication, submission, tracking, administration, persistence, security controls, and tests are implemented.

Progress split



Evidence used

- Code: 20+ Java classes across config, DAO, model, service, UI, and util packages.
- Data: PostgreSQL schema with six tables, constraints, indexes, seed users, tickets, hypes, notes, and history.
- Quality: Maven/JUnit suite covering authentication, submissions, hype, password rules, validation, and anonymity regression.

Progress Timeline

From the zeroth presentation to the current review

1

Zeroth presentation

Approved objective, architecture, privacy boundary, and interface direction.

2

UI designed

FlatLaf Swing entry point and student/admin dashboard frames are wired.

3

Database created

PostgreSQL schema, seed data, indexes, foreign keys, and checks are present.

4

Authentication completed

Password login, BCrypt, Google OAuth/PKCE path, domain gate, and session context.

5

Student module completed

Submission form, receipt vault lookup, personal list, detail history, and hype action.

6

Admin module progress

Queue, status change, resolution notes, history view, and role guard are present.

7

Current stage

Integration hardening, reporting/analytics, polish, wider test coverage, and documentation closure.

Completed Work | Core Modules

Implemented classes, data path, screens, and status

Module	Classes	Tables	Screens / behavior	Status
Authentication	AuthService, GoogleOAuthService, UserDAO, LoginFrame, ValidationUtil, PasswordUtil	users	Password + OAuth entry; domain gate; session set	Complete / OAuth configuration dependent
Student dashboard	StudentDashboardFrame, SubmissionFormPanel, Submission, User	submissions, receipts, hype	Personal table, new submission, details, hype	Complete core flow
Issue + suggestion	SubmissionService, DatabaseSubmissionService, SubmissionDAO	submissions	Form writes ISSUE or SUGGESTION with category/location/priority	Complete
Status tracking	SubmissionHistory, ResolutionNote, SubmissionDAO, service	submission_history, resolution_notes	History and note display in student/admin detail views	Complete core flow
Database connectivity	DatabaseConfig, DAO classes	All six tables	JDBC DriverManager, prepared statements, transactions	Complete; environment configuration required

Implementation notes

- Anonymity: Submission rows do not store student_id. StudentReceiptDAO is the private ownership mapping used for My Submissions.
- Integrity: SubmissionDAO uses transactions for createWithReceipt and updateStatusWithHistory; database uniqueness blocks duplicate receipts and hypes.
- UI: Main applies FlatDarkLaf. LoginFrame routes authenticated STUDENT and ADMIN roles to separate frames.
- Caveat: Several classes describe themselves as minimal skeletons, and the dashboard UI is functional but not yet the final polished interface.

Completed Work | Admin, Hype, Security

The operational spine of the system

Admin dashboard

AdminDashboardFrame loads all submissions, changes status through the five-state list, adds resolution notes, views history, refreshes, and logs out.

Hype / trending

HypeService and SubmissionHypeDAO add, remove, count, and detect a student's hype. SubmissionDAO sorts by hype_count then created_at.

RBAC

AuthService and SessionContext gate student/admin screens. AdminDashboardFrame rejects unauthenticated or non-admin sessions; student frame rejects admin sessions.

Security

ValidationUtil enforces @tkmce.ac.in and minimum password length. PasswordUtil uses BCrypt. DAO queries use PreparedStatement. OAuth uses state + PKCE.

Testing

Existing JUnit tests cover AuthService, DatabaseSubmissionService, HypeService, PasswordUtil, ValidationUtil, and AdminAnonymityRegressionTest.

Source-of-truth status

This 70% score is a project review estimate: the primary user journey works, while reporting, analytics, notification, polish, and broader integration closure remain.

Pending Work | Remaining 30%

Prioritised closure plan

Pending feature	Responsible	Priority	Estimate	Definition of done
Report generation	Nirmal	High	3-5 days	CSV/export service and admin action
Analytics	Ringshit	High	3-5 days	Counts, trends, SLA-style metrics
Notifications	Adil	Medium	4-6 days	Status-change feedback and delivery design
History improvements	Nafee	Medium	2-3 days	Richer student timeline and edge cases
UI polishing	Ringshit	High	4-6 days	Replace placeholder layouts, validation states, responsive sizing
Testing	Rehandeep	High	4-6 days	DAO coverage, integration setup, failure paths
Bug fixing	All	High	Ongoing	Run full suite and resolve DB/config/runtime issues
Documentation	Nafee	Medium	2-3 days	Keep diagrams, setup, and speaking evidence current

Completion gate

The remaining work is complete when every pending item has an implementation or accepted design decision, a test or verification note, and a presentation-ready screen or output.

Team Responsibility | Ringshit

One-page ownership brief and speaking plan

Student experience and analytics lead

Assigned module	Student experience and analytics lead
Main files / classes	StudentDashboardFrame, SubmissionFormPanel, SubmissionService integration, analytics/report UI
Progress	Student flow is core-complete; polish and analytics remain
Remaining tasks	Finish UI states, filters, trending/analytics view, and demo student journey

2-3 MINUTE SPEAKING PART

Explain the student problem, anonymous submission, hype, and how the dashboard turns reports into visible campus priorities.

Evidence to show

StudentDashboardFrame and SubmissionFormPanel; demonstrate new submission, My Submissions, and hype.

Team Responsibility | Nirmal

One-page ownership brief and speaking plan

Reporting and admin workflow lead

Assigned module	Reporting and admin workflow lead
Main files / classes	AdminDashboardFrame, report requirements, admin queue and export design
Progress	Admin triage core is implemented; reporting remains
Remaining tasks	Implement CSV/report generation, admin filters, and review metrics

2-3 MINUTE SPEAKING PART

Explain how administrators read the queue, change lifecycle status, attach resolution notes, and use reports for accountability.

Evidence to show

AdminDashboardFrame; demonstrate queue, status transition, resolution note, and planned report.

Team Responsibility | Rehandeep

One-page ownership brief and speaking plan

Persistence and quality lead

Assigned module	Persistence and quality lead
Main files / classes	UserDAO, SubmissionDAO, DatabaseConfig integration, database tests
Progress	Database foundation is complete; test depth remains
Remaining tasks	Add DAO integration coverage, schema verification, and failure-path tests

2-3 MINUTE SPEAKING PART

Defend JDBC, prepared statements, transactions, six tables, indexes, and why persistence is isolated behind DAO classes.

Evidence to show

sql/schema.sql plus SubmissionDAO; point to transaction boundaries, indexes, and prepared statements.

Team Responsibility | Adil

One-page ownership brief and speaking plan

Security and notification lead

Assigned module	Security and notification lead
Main files / classes	Auth/session review, security acceptance, notification design and UI feedback
Progress	Authentication/security controls are present; notifications remain
Remaining tasks	Design notification events, status feedback, and security regression checklist

2-3 MINUTE SPEAKING PART

Explain role boundaries, domain restriction, BCrypt, OAuth state/PKCE, and the next step for user notifications.

Evidence to show

AuthService, SessionContext, ValidationUtil, and LoginFrame; explain role protection and notification gap.

Team Responsibility | Nafee

One-page ownership brief and speaking plan

Authentication, privacy, and documentation lead

Assigned module	Authentication, privacy, and documentation lead
Main files / classes	AuthService, GoogleOAuthService, PasswordUtil, ValidationUtil, SessionContext, StudentReceiptDAO
Progress	Authentication and anonymity boundary are implemented
Remaining tasks	Harden history edge cases, consolidate docs, and lead final integration review

2-3 MINUTE SPEAKING PART

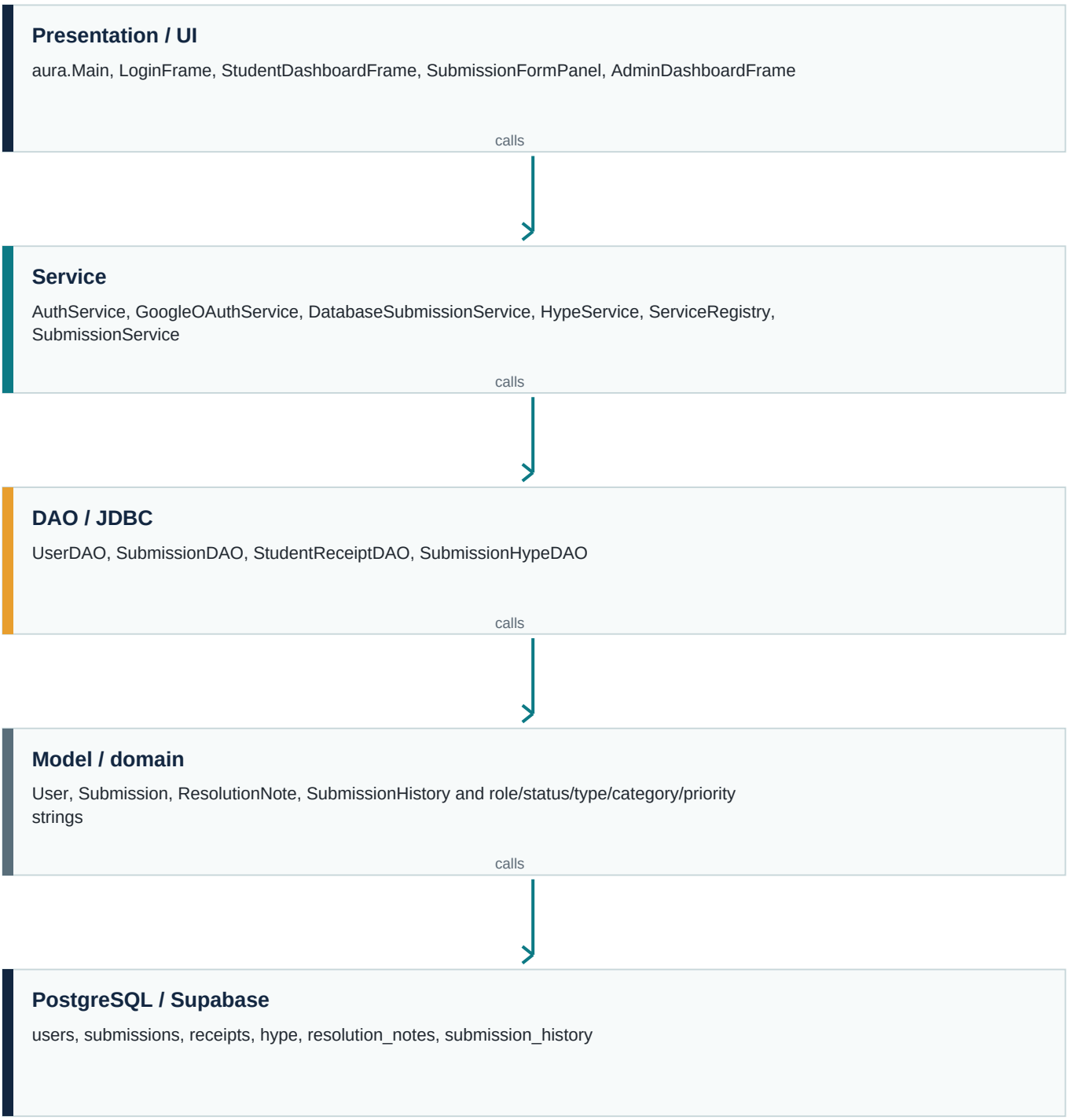
Explain the login journey, session routing, decoupled anonymity vault, and why admins can resolve a ticket without knowing its author.

Evidence to show

GoogleOAuthService, StudentReceiptDAO, SECURITY.md; show OAuth path and the no-student_id rule.

Complete Code Architecture

MVC-style presentation, service orchestration, DAO persistence



Dependency rule

Each layer depends downward. UI never executes SQL; services own business rules and session checks; DAO owns JDBC and PreparedStatement work; models carry data between boundaries.

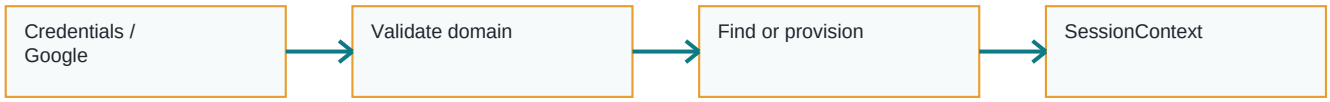
Architecture and Workflow Diagrams

Simplified diagrams derived from the current code

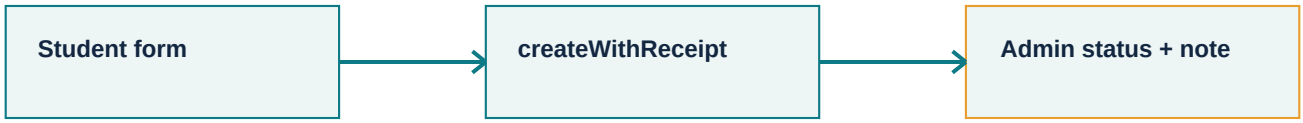
Request path



Login flow



Student submit -> admin resolve



Status state machine



Database interaction

DatabaseConfig loads db.properties or environment variables, loads org.postgresql.Driver, and returns a TLS-capable JDBC connection. DAO methods use try-with-resources; status updates with history use a transaction and rollback on failure.

Class Documentation | Models and Utilities

Important classes in the actual source tree

Class	Purpose / attributes	Key methods	Called by
User	Domain user; id, name, email, passwordHash, role	getters/setters, isAdmin, isStudent	AuthService, UserDAO, SessionContext
Submission	Anonymous ticket data and hype count	getters/setters for title, type, category, status, photo	SubmissionService, SubmissionDAO, both dashboards
ResolutionNote	Admin resolution text linked to a submission	getters/setters for ids, note, createdAt	SubmissionDAO, dashboards
SubmissionHistory	Old/new status audit record	getters/setters for transition and actor	SubmissionDAO, dashboards
AuthService	Credential/OAuth login, registration, logout, RBAC checks	authenticate, registerStudent, loginWithGoogle, loginMock, logout	LoginFrame, dashboard guards
SessionContext	Volatile process-local active session	set/get current user, role checks, clear	AuthService and UI guards
ValidationUtil	Institutional email/password rules	isValidTkmceEmail, normalizeEmail, isValidPassword	AuthService, UserDAO, tests

Relationships

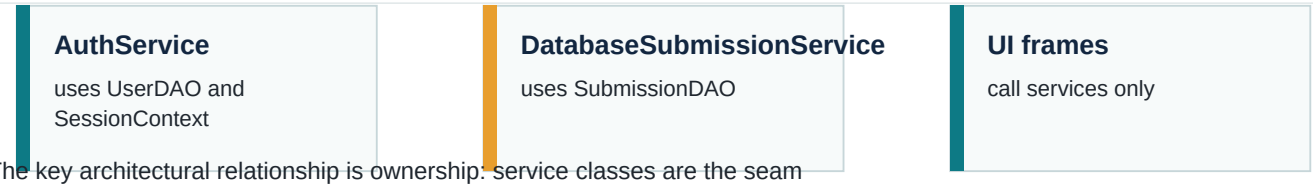
AuthService creates or retrieves User and places it in SessionContext. SubmissionService coordinates Submission, ResolutionNote, and SubmissionHistory through SubmissionDAO. StudentReceiptDAO links the current student to an otherwise anonymous Submission.

Class Documentation | Services, DAO, and UI

Who calls what and where database access stops

Class	Purpose	Key methods	Called by
UserDAO	CRUD users; maps ResultSet to User	save, findById, findByEmail, update, delete	AuthService
SubmissionDAO	Submission CRUD, receipts, history, notes, hype aggregation	save, findById, findAll, createWithReceipt, updateStatusWithHistory	DatabaseSubmissionService
StudentReceiptDAO	Private receipt vault lookup	createReceipt, findSubmissionIdsByStudent	student tracking service path
SubmissionHypeDAO	Single-vote persistence and counts	addHype, removeHype, count, hasStudentHyped	HypeService
DatabaseSubmissionService	Concrete SubmissionService and error translation	create/list/find/update/note/history operations	ServiceRegistry, dashboards
LoginFrame	Password, OAuth, mock login entry	passwordLogin, googleLogin, openDashboard	Main
StudentDashboardFrame	Student list and actions	loadSubmissions, openSubmissionDialog, showDetails	LoginFrame
AdminDashboardFrame	Admin queue and controls	loadData, status, note, history	LoginFrame

UML relationship sketch



The key architectural relationship is ownership: service classes are the seam between user actions and persistence. UI classes do not import java.sql.*; DAO classes are the only direct SQL boundary.

Database Documentation

ER model, keys, constraints, and sample records

ER diagram (simplified)



Keys and sample data

Table	Primary key	Seed evidence	Important constraint
users	user_id	6 seeded: 4 STUDENT, 2 ADMIN	@tkmce.ac.in check
submissions	submission_id	5 tickets: 104, 103, 98, 89, 85	5-state status check
receipts	receipt_id	5 private ownership mappings	unique student + submission
submission_hype	hype_id	10 seeded votes	unique submission + student
resolution_notes	note_id	3 admin notes	submission/admin FKs
submission_history	history_id	6 lifecycle transitions	submission/actor FKs

Folder Structure

Actual repository tree and purpose of each area

AURA/

```
|-- AGENT.md                engineering and security rules
|-- pom.xml                 Java 17, Maven dependencies, test runner
|-- mvp_dashboard.html      visual MVP reference
|-- docs/                   architecture, PRD, roadmap, security, tests
|   |-- ARCHITECTURE.md     layer rules and diagrams
|   |-- DATABASE.md         ER model and data dictionary
|   |-- SECURITY.md         threat model and anonymity boundary
|   '-- TESTING.md          verification plan
|-- sql/                    deployable persistence definition
|   |-- schema.sql          tables, constraints, indexes
|   '-- seed.sql            sample users, tickets, votes, history
|-- src/main/java/aura/     application source
|   |-- config/             DatabaseConfig and OAuthConfig
|   |-- dao/                JDBC persistence classes
|   |-- model/              User and submission domain objects
|   |-- service/            business orchestration and registry
|   |-- ui/                 Swing login, student, admin screens
|   '-- util/               validation, BCrypt, process session
|-- src/main/resources/     db.properties.example and OAuth example
'-- src/test/java/aura/     JUnit service, security, and utility tests
```

Reading guide

Start at Main and LoginFrame. Follow the service call to the DAO. Then inspect schema.sql to see the relational contract and the test classes to see the acceptance evidence.

Presentation Speaking Script

A shared sequence for the second review

Recommended order

- 1. Nafee: Set the context: objective, login/security, anonymity vault, and the 70/30 position.
- 2. Ringshit: Demonstrate the student experience: submit, view personal tickets, hype, and inspect history.
- 3. Rehandeep: Defend the database and DAO layer: schema, keys, indexes, prepared statements, and transactions.
- 4. Nirmal: Demonstrate the admin queue: status progression, resolution note, history, and reporting gap.
- 5. Adil: Close with security acceptance, notification plan, testing gaps, and the remaining work checklist.

Common technical phrases to use

- The UI calls services; it never touches JDBC directly.
- Anonymity is enforced by the schema because submissions has no student_id.
- The receipt vault lets a student find their own ticket without exposing ownership to the admin queue.
- PreparedStatement, foreign keys, uniqueness constraints, and transactions protect integrity.
- The 70% claim describes the working core; the 30% is closure work around reporting, analytics, notifications, polish, testing, and documentation.

Final Review Checklist

Use this page during the rehearsal and before submission

Completed features

[ ] Password login and institutional email validation

[ ] Google OAuth configuration tested

[ ] Student submission and receipt flow

[ ] Hype and duplicate-vote rule

[ ] Admin status, notes, and history

[ ] Database schema, seed data, and anonymity boundary

Pending features

[ ] Report generation

[ ] Analytics and filters

[ ] Notifications

[ ] History improvements

[ ] Final UI polish

Quality and delivery

[ ] Full Maven test run with configured database

[ ] Bug list triaged with owners

[ ] DAO and integration tests expanded

[ ] README/setup and diagrams updated

[ ] Each member rehearsed a 2-3 minute section

[ ] Demo data and presentation screenshots ready

FINAL MESSAGE

AURA already demonstrates a complete academic core: secure access, anonymous reporting, structured persistence, role separation, and auditable resolution. The next review should show the core live and frame the remaining 30% as a disciplined closure plan.

How To Use This Manual

A guided study path for a team member or examiner new to AURA

Purpose

This manual explains how the AURA project was built, how the code is connected, where each feature lives, how the database protects the workflow, and how to reproduce the development process. Read it in order once, then use the member chapters and code maps as revision notes before the presentation or viva.

Recommended study sequence

1. **Understand the problem**

   AURA converts campus complaints and suggestions into anonymous, trackable work items.

2. **Learn the layers**

   UI starts the action, services apply rules, DAOs execute SQL, and PostgreSQL stores the result.

3. **Trace one workflow**

   Follow login, student submission, receipt creation, admin status update, and history recording.

4. **Open the source**

   Use the file/class map to locate each method described in the manual.

5. **Rehearse ownership**

   Each member page contains their responsibility, evidence to show, and a speaking script.

6. **Verify the build**

   Run the Maven tests and, when configured, run the application against the PostgreSQL database.

> **Important source note**
>
> The repository implements Java Swing with FlatLaf, not JavaFX. This manual describes the implementation that exists in the source tree. Architecture and roadmap documents may describe future or approved components; those are marked as planned when they are not present in code.

Step 1 | Set Up The Project

What a new developer does before changing code

Required environment

- Java: Java 17 is configured in pom.xml as both compiler source and target.
- Maven: Maven resolves PostgreSQL JDBC, FlatLaf, BCrypt, Gson, and JUnit Jupiter.
- Database: A PostgreSQL/Supabase database must contain sql/schema.sql and optionally sql/seed.sql.
- Configuration: Copy db.properties.example and oauth.properties.example into local properties files; keep secrets out of Git.
- Test isolation: src/test/resources contains the test configuration example. Tests that use the database require configured connection values.

Commands

Build and test

```
mvn test
# compile production code only
mvn -DskipTests package
# run the desktop entry point
mvn exec:java -Dexec.mainClass=aura.Main
```

First inspection order

Open pom.xml to learn dependencies, Main.java to find the entry point, LoginFrame.java to see the first screen, AuthService.java to see authentication rules, DatabaseSubmissionService.java to see the service implementation, SubmissionDAO.java to see SQL, and schema.sql to see the database contract.

Expected result

A successful setup compiles Java 17 sources, discovers the six JUnit test classes, and can start the Swing login window when local configuration is available.

Step 2 | Read The Project Layers

The dependency direction that keeps the system explainable

The AURA request path

User action

LoginFrame, StudentDashboardFrame, AdminDashboardFrame

Business decision

AuthService, HypeService, DatabaseSubmissionService

Persistence operation

UserDAO, SubmissionDAO, StudentReceiptDAO, SubmissionHypeDAO

Stored state

users, submissions, student_submission_receipts, submission_hype, resolution_notes, submission_history

How to decide where new code belongs

- UI: Only collect input, show output, respond to clicks, and route to a service.
- Service: Validate roles, coordinate multiple DAO calls, enforce business rules, and translate failures.
- DAO: Open a connection, use a parameterized SQL statement, map rows to models, and close resources.
- Model: Represent the data transferred between layers; do not perform database work.
- Config/util: Provide shared infrastructure such as connection configuration, OAuth properties, password hashing, validation, and session state.

Step 3 | Understand The Data Model

The objects that move through the application

Model class	Attributes	Why it exists
User	userId, name, email, passwordHash, role	Account and RBAC identity
Submission	submissionId, title, description, type, category, location, priority, status, photoUrl, hypeCount	Anonymous public ticket
ResolutionNote	noteId, submissionId, adminId, note, createdAt	Official admin response
SubmissionHistory	historyId, submissionId, oldStatus, newStatus, changedBy, changedAt	Lifecycle audit entry

Important design decision: identity is separated

Submission deliberately has no studentId attribute. The student-to-submission relationship is held by StudentReceiptDAO and the student_submission_receipts table. This allows a student to retrieve their own tickets while the administrative submission queue sees only anonymous ticket data.

SubmissionDAO creates the anonymous row and private receipt in one transaction

```
connection.setAutoCommit(false);
insertSubmission.executeUpdate();
submission.setSubmissionId(keys.getInt(1));
receipt.setInt(1, studentId);
receipt.setInt(2, submission.getSubmissionId());
connection.commit();
```

Viva answer

The receipt is not a display trick. It is a separate relational mapping, so the anonymity boundary survives even when the database is inspected directly.

Step 4 | Authentication From Click To Session

Detailed login implementation

Password login path

1. LoginFrame.passwordLogin reads the email field and password field.
2. AuthService.authenticate rejects null values and calls validateCredentials.
3. ValidationUtil checks the official @tkmce.ac.in pattern and an eight-character minimum password.
4. UserDAO.findByEmail uses a PreparedStatement against users.
5. PasswordUtil.verifyPassword compares the plaintext input with the BCrypt hash.
6. AuthService places the User in SessionContext when verification succeeds.
7. LoginFrame.openDashboard routes ADMIN to AdminDashboardFrame and other users to StudentDashboardFrame.

AuthService.authenticate

```
if (email == null || password == null) return null;
if (!validateCredentials(email, password)) return null;
User user = userDAO.findByEmail(email);
if (user == null || !PasswordUtil.verifyPassword(password,
        user.getPasswordHash())) return null;
SessionContext.setCurrentUser(user);
```

Google OAuth path

loginWithGoogle delegates browser authentication to GoogleOAuthService. The service starts a loopback HttpServer, creates state and PKCE values, opens the browser, receives the callback, exchanges the authorization code, retrieves verified profile data, and returns GoogleUserInfo. AuthService applies the institutional domain check, finds or provisions a STUDENT, and stores the session.

Local development

OAuthConfig supports a development mock mode. loginMock creates a demo STUDENT or ADMIN only when development mode is enabled; this is useful for UI work but is not a production authentication path.

Step 5 | Student Creates A Submission

The complete issue/suggestion workflow

1. **Open**
   After successful login, StudentDashboardFrame verifies the current session is a student and loads the personal list.
2. **Fill**
   SubmissionFormPanel collects title, description, type, category, location, priority, and optional photo URL.
3. **Validate**
   The panel requires the core fields before it constructs a Submission object.
4. **Call service**
   The form callback calls ServiceRegistry.getSubmissionService().createSubmission(submission, user.getUserId()).
5. **Write atomically**
   DatabaseSubmissionService delegates to SubmissionDAO.createWithReceipt, which inserts the anonymous submission and receipt together.
6. **Refresh**
   The dashboard reloads the student IDs from the receipt vault, retrieves matching Submission objects, and refreshes the JTable.
7. **Support**
   HypeService.addHype records a single student vote and the dashboard shows the updated count.

Service interface called by the UI

```
Submission createSubmission(Submission s, int studentId);
List<Submission> listSubmissionsByStudent(int studentId);
void updateStatus(int submissionId, String newStatus,
                  int changedBy);
```

What to demonstrate

Use the demo student, create an ISSUE, refresh the table, select it, add Hype, open details, and explain that the admin queue can see the ticket but not its author.

Step 6 | Admin Resolves A Submission

The operational lifecycle and audit trail

Admin workflow

- Guard: AdminDashboardFrame constructor rejects a session that is not authenticated as ADMIN.
- Load: ServiceRegistry returns the SubmissionService; listAllSubmissions loads the shared queue sorted by hype and recency.
- Select: The administrator chooses a row from the JTable and reads its submission ID.
- Transition: The Change Status control chooses PENDING, ASSIGNED, IN_PROGRESS, RESOLVED, or REJECTED.
- Record: DatabaseSubmissionService obtains the old status and asks SubmissionDAO.updateStatusWithHistory to update the ticket and insert the audit record in one transaction.
- Explain: Add Resolution Note writes the admin’s note through SubmissionDAO and the history view reads both notes and transitions.
- Logout: AuthService.logout clears SessionContext and returns to LoginFrame.

Transactional status update

```
connection.setAutoCommit(false);
update.setString(1, newStatus);
update.setInt(2, id);
update.executeUpdate();
history.setString(2, oldStatus);
history.setString(3, newStatus);
history.setInt(4, changedBy);
history.executeUpdate();
connection.commit();
```

Why history is separate

The submissions row answers ’what is the current state?’ The submission_history rows answer ’how did it reach this state and which administrator changed it?’ Keeping both supports a fast dashboard and an audit trail.

Failure behavior

If either the status update or history insert fails, the DAO rolls back the transaction and rethrows the SQLException. The ticket cannot silently change without its audit record.

Step 7 | Database Build Manual

How the relational contract supports the application

Run order

1. Create or connect to a PostgreSQL/Supabase project.
2. Run sql/schema.sql. It creates users first, submissions second, then dependent receipts, hypes, notes, and history.
3. Run sql/seed.sql for the presentation dataset: six users, five tickets, receipts, hypes, notes, and transitions.
4. Copy the database example properties into a local db.properties file and set db.url, db.user, db.password, and db.driver.
5. Start the application or test suite. DatabaseConfig loads the properties from the classpath and calls DriverManager.getConnection.

Table responsibilities

Table	Purpose
users	accounts, roles, BCrypt hashes, institutional email check
submissions	anonymous issue/suggestion content and current status
student_submission_receipts	private student ownership mapping
submission_hype	crowd support with duplicate-vote uniqueness
resolution_notes	official administrator response
submission_history	old/new state transitions and actor

Step 8 | JDBC And DAO Code

How Java turns an action into a safe SQL operation

Every DAO method follows this pattern

- 1. Prepare: Declare a SQL string with ? placeholders; never concatenate user input into SQL.
- 2. Connect: Use DatabaseConfig.getConnection() inside try-with-resources.
- 3. Bind: Set values using setInt, setString, or the appropriate PreparedStatement method.
- 4. Execute: Call executeQuery for reads or executeUpdate for writes.
- 5. Map: Convert ResultSet columns to a model object or collection.
- 6. Close: try-with-resources closes the connection, statement, and result set automatically.
- 7. Translate: The service layer catches SQL failures and exposes a runtime/application-level failure to the UI.

UserDAO.findByEmail

```
String sql = "SELECT user_id, name, email,"
        + " password_hash, role FROM users WHERE email = ?";
try (Connection connection = DatabaseConfig.getConnection();
     PreparedStatement statement = connection.prepareStatement(sql)) {
    statement.setString(1, ValidationUtil.normalizeEmail(email));
    try (ResultSet results = statement.executeQuery()) {
        return results.next() ? mapUser(results) : null;
    }
}
```

DAO ownership map

UserDAO owns user CRUD. SubmissionDAO owns submissions plus the compound receipt transaction, status history, and resolution notes. StudentReceiptDAO owns vault lookups. SubmissionHypeDAO owns vote operations. No UI class should import java.sql.* or call these classes directly.

Examiner checkpoint

PreparedStatement protects against SQL injection; foreign keys protect relationships; unique constraints protect one receipt and one hype per student-ticket pair.

Step 9 | Security And Privacy

Controls implemented in code and SQL

Threat / rule	Implementation	What it guarantees
Domain gate	ValidationUtil	Regex accepts only @tkmce.ac.in; schema adds CHECK constraint
Password protection	PasswordUtil + UserDAO	BCrypt hash is persisted; plaintext is not stored
SQL injection	All DAO classes	PreparedStatement placeholders and bound parameters
OAuth callback	GoogleOAuthService	Loopback server, state comparison, PKCE challenge, verified email
Session control	SessionContext + dashboard constructors	Current user and role are checked before screen access
Anonymity	SubmissionDAO + StudentReceiptDAO + schema	No student_id in submissions; ownership exists only in vault

The anonymity proof

An admin listing submissions receives Submission objects with title, description, category, location, priority, status, photo URL, and hype count. There is no student ID field to display. A student retrieves personal tickets by asking StudentReceiptDAO for submission IDs using the authenticated student ID, then loading the public submission records.

ValidationUtil institutional email rule

```
private static final Pattern TKMCE_EMAIL =
    Pattern.compile("^[A-Z0-9._%+-]+@tkmce\\.ac\\.in$",
                    Pattern.CASE_INSENSITIVE);
```

Security limitation to discuss honestly

The repository contains strong design and regression coverage, but full database integration and production OAuth verification still depend on local credentials and environment configuration.

Step 10 | Testing Manual

How the project checks behavior instead of relying only on the demo

Test class	Current suite	What to learn from it
AuthServiceTest	Three tests	Credential validation and authentication behavior
DatabaseSubmissionServiceTest	Three tests	Submission creation, listing, and service operations
HypeServiceTest	One test	Hype service behavior
PasswordUtilTest	Two tests	BCrypt hashing and verification
ValidationUtilTest	Two tests	Institutional email and password rules
AdminAnonymityRegressionTest	One test	Admin path cannot expose student identity

Test workflow

- Run: mvn test from the repository root.
- Read: Surefire reports are written under target/surefire-reports.
- Diagnose: A failure may mean a code defect, missing database properties, or an environment mismatch.
- Extend: For remaining work, add tests before calling report, analytics, notification, and UI improvements complete.
- Defend: Explain the behavior under test, the input, the expected output, and which layer owns the rule.

Representative JUnit shape

```
@Test
void invalidEmailIsRejected() {
    assertFalse(ValidationUtil.isValidTkmceEmail(
        "person@gmail.com"));
}
```

Class Map | Configuration And Utilities

Where to open the code when studying infrastructure

File	Package	Study explanation
Main.java	aura	Applies FlatDarkLaf and opens LoginFrame on the Swing event thread.
DatabaseConfig.java	aura.config	Loads db.properties/environment values, loads the driver, returns JDBC connections.
OAuthConfig.java	aura.config	Loads OAuth settings, enforced domain, server port, and development mode.
PasswordUtil.java	aura.util	Hashes and verifies passwords with BCrypt work factor 12.
ValidationUtil.java	aura.util	Checks institutional email, normalizes it, and checks password length.
SessionContext.java	aura.util	Stores the active User in a volatile process-local field and provides role checks.

Objects created during startup and login

- Main: creates LoginFrame through SwingUtilities.invokeLater
- LoginFrame: creates AuthService, text fields, password field, and buttons
- AuthService: creates GoogleOAuthService and UserDAO through its default constructor
- Successful login: creates or retrieves User and writes it to SessionContext
- Dashboard: creates ServiceRegistry-backed submission service and UI controls

Class Map | DAO And Models

The persistence objects and their relationships

Class	Package	Responsibilities and relationships
UserDAO	aura.dao	save, findById, findByEmail, update, delete; maps users table rows to User.
SubmissionDAO	aura.dao	save, findById, findByIds, findAll, updateStatus, createWithReceipt, history, notes, delete.
StudentReceiptDAO	aura.dao	Creates and retrieves private receipt mappings.
SubmissionHypeDAO	aura.dao	Adds/removes hypes and answers count/has-hyped queries.
User	aura.model	Identity and role fields; helper methods isAdmin/isStudent.
Submission	aura.model	Ticket fields; no author field by design.
ResolutionNote	aura.model	Administrative note data.
SubmissionHistory	aura.model	Status transition data and actor ID.

Call relationship

AuthService -> UserDAO -> User. DatabaseSubmissionService -> SubmissionDAO -> Submission, ResolutionNote, SubmissionHistory. StudentReceiptDAO -> submission IDs -> SubmissionDAO. HypeService -> SubmissionHypeDAO -> submission_hype.

Study tactic

For each DAO method, identify the SQL statement, input parameters, returned object, table touched, and whether a transaction is required. That five-question pattern covers most of the viva.

Class Map | Services And UI

The classes that turn actions into workflows

Class	Purpose	Called from
AuthService	Authentication, registration, Google login, mock login, logout, current-user checks	LoginFrame
GoogleOAuthService	Authorization code + loopback callback + PKCE + profile retrieval	AuthService
DatabaseSubmissionService	Concrete SubmissionService; wraps DAO calls and converts failures	ServiceRegistry, both dashboards
HypeService	Hype actions and duplicate handling	StudentDashboardFrame
ServiceRegistry	Provides the shared submission service	StudentDashboardFrame, AdminDashboardFrame
LoginFrame	Collects credentials and routes roles	Main
StudentDashboardFrame	Personal submissions, new form, hype, details, logout	LoginFrame
AdminDashboardFrame	All submissions, status, notes, history, logout	LoginFrame
SubmissionFormPanel	Collects and validates submission fields	StudentDashboardFrame

Why ServiceRegistry exists

The UI should not construct a new collection of DAOs or duplicate business rules. ServiceRegistry centralizes the concrete DatabaseSubmissionService used by both dashboards. It is a small composition point, not a second business layer.

Member Study Sheet | Mohammed Nafih

Detailed ownership, source files, and presentation preparation

Authentication, privacy, and session lead

Main assignment	Authentication, privacy, and session lead
Files to study	AuthService.java, GoogleOAuthService.java, PasswordUtil.java, ValidationUtil.java, SessionContext.java, StudentReceiptDAO.java
Speaking responsibility	Explain institutional login, BCrypt, OAuth state/PKCE, role session, and why the receipt vault protects anonymity.
Remaining work	Finish history edge cases, OAuth/database integration checks, documentation, and final integration review.

Study checklist

- [ ] Open every listed file and identify its package.
- [ ] Trace at least one method call from UI to database.
- [ ] Explain one validation or security rule.
- [ ] Demonstrate one feature using seeded data.
- [ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Member Study Sheet | Muhammed Rinshid

Detailed ownership, source files, and presentation preparation

Student experience and product-flow lead

Main assignment	Student experience and product-flow lead
Files to study	StudentDashboardFrame.java, SubmissionFormPanel.java, Submission.java, student-facing service integration
Speaking responsibility	Demonstrate the student journey: login, create issue/suggestion, view personal submissions, inspect updates, and hype a ticket.
Remaining work	Polish form validation and dashboard states; help implement analytics/trending presentation.

Study checklist

- [ ] Open every listed file and identify its package.
- [ ] Trace at least one method call from UI to database.
- [ ] Explain one validation or security rule.
- [ ] Demonstrate one feature using seeded data.
- [ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Member Study Sheet | Nirmal

Detailed ownership, source files, and presentation preparation

Administration, reporting, and operational workflow lead

Main assignment	Administration, reporting, and operational workflow lead
Files to study	AdminDashboardFrame.java, DatabaseSubmissionService.java, SubmissionService.java, admin workflow documentation
Speaking responsibility	Demonstrate queue review, status transitions, resolution notes, history, and the need for report generation.
Remaining work	Implement CSV/report output, admin filters, metrics, and report tests.

Study checklist

- [ ] Open every listed file and identify its package.
- [ ] Trace at least one method call from UI to database.
- [ ] Explain one validation or security rule.
- [ ] Demonstrate one feature using seeded data.
- [ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Member Study Sheet | Rahandeep

Detailed ownership, source files, and presentation preparation

Database and DAO quality lead

Main assignment	Database and DAO quality lead
Files to study	UserDAO.java, SubmissionDAO.java, SubmissionHypeDAO.java, DatabaseConfig.java, sql/schema.sql, sql/seed.sql
Speaking responsibility	Defend the six-table schema, prepared statements, transactions, indexes, foreign keys, and seeded demo records.
Remaining work	Expand DAO integration coverage and verify failure paths against the configured database.

Study checklist

[ ] Open every listed file and identify its package.

[ ] Trace at least one method call from UI to database.

[ ] Explain one validation or security rule.

[ ] Demonstrate one feature using seeded data.

[ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Member Study Sheet | Adil Rahman

Detailed ownership, source files, and presentation preparation

Security, testing, and completion lead

Main assignment	Security, testing, and completion lead
Files to study	AdminAnonymityRegressionTest.java, AuthServiceTest.java, HypeServiceTest.java, PasswordUtilTest.java, ValidationUtilTest.java, security/testing documentation
Speaking responsibility	Explain what the tests prove, the security checklist, remaining notification work, and the 70/30 completion plan.
Remaining work	Add broader regression/integration tests, design notifications, triage bugs, and coordinate final rehearsal.

Study checklist

- [ ] Open every listed file and identify its package.
- [ ] Trace at least one method call from UI to database.
- [ ] Explain one validation or security rule.
- [ ] Demonstrate one feature using seeded data.
- [ ] State one completed item and one remaining item without overstating progress.

Presentation close

End by connecting your module to the next layer. AURA is a team system: a screen action is only complete when the service rule, DAO operation, database constraint, and user-visible result agree.

Remaining Work | Step-By-Step Closure

How to turn the current 70% into a defensible final release

#	Workstream	Step-by-step action
1	Freeze current behavior	Run mvn test, record the baseline, and keep the working demo seed data.
2	Complete admin reporting	Add a report service/output, connect it to an admin control, and test the exported result.
3	Add analytics	Calculate counts by status/category/priority and expose them in a dashboard view.
4	Design notifications	Decide whether notifications are in-app, email, or status feedback; implement the chosen minimal path.
5	Polish UI	Replace placeholder layouts, add validation messages, loading/error states, and consistent sizing.
6	Expand tests	Cover DAO failure paths, status transition rules, receipt ownership, and integration configuration.
7	Run a bug sweep	Test invalid input, missing configuration, duplicate hype, unauthorized screens, and rollback behavior.
8	Update evidence	Refresh README, diagrams, screenshots, manual, and each member's presentation notes.

Definition of done for the next review

Every presented feature should have a visible path in the UI, a service method that owns its rule, a DAO/database operation where persistence is needed, a test or repeatable verification procedure, and an assigned member who can explain it line by line.

Quick Viva Revision

Questions an unfamiliar person should be able to answer

What is AURA?	A campus governance platform for anonymous issues and suggestions with student tracking and administrator resolution.
Why is the submission anonymous?	The submissions table and Submission model contain no student_id; identity is isolated in the student receipt vault.
Where does SQL live?	Only in aura.dao classes, using DatabaseConfig connections and PreparedStatement.
How does a student find their own tickets?	StudentReceiptDAO finds IDs for the authenticated student, then SubmissionDAO loads the anonymous submissions.
How is a password protected?	PasswordUtil hashes with BCrypt before UserDAO saves it and verifies it during authentication.
How is an administrator identified?	User.Role is stored as ADMIN, SessionContext exposes isAdmin, and AdminDashboardFrame checks it before opening.
How is a status change auditable?	SubmissionDAO updates submissions and inserts submission_history in one transaction.
What is complete?	Core authentication, submission, student tracking, hype, admin resolution, persistence, security controls, and baseline tests.
What remains?	Reporting, analytics, notifications, UI polish, wider tests, bug fixing, and documentation closure.
What should never be claimed?	Do not claim JavaFX, report generation, analytics, or production OAuth validation as complete unless those code paths are actually added and tested.